

Progetto Reti

Progetto Reti.....	1
Introduzione.....	2
Scelte Progettuali.....	2
Utilizzo Code Management System (github).....	2
Modifiche successive.....	3
Active VS Passive.....	3
Active.....	3
Codice.....	4
Passive.....	5
Codice.....	6
Firewall.....	7
► SERVER.....	8
Strutture utilizzate.....	8
Commands.....	8
Session.....	8
Array COMMANDS.....	9
Ricerca/Esecuzione handler.....	9
Funzionamento.....	9
- Creazione Socket server.....	9
Conversioni.....	10
- Struttura generale Server.....	11
accept() + pthread_create().....	11
Thread: handle_client.....	13
Handlers.....	13
Logging.....	15
Scelte.....	16
Server multithread.....	16
Implementazione logging (nel server).....	16
Gestione struct con puntatori a funzione.....	16
Credenziali.....	16
► CLIENT.....	17
Funzionamento.....	17
- Creazione Socket Client.....	17
- Struttura generale Client.....	18
Struttura While.....	18
Gestione comandi (dentro while).....	18
Gestione Active VS Passive.....	19
Problematiche.....	20
Buffer overflow per LIST.....	20
Messaggio singolo/doppio.....	20
Credenziali.....	20

Introduzione

FTP (File Transfer Protocol) e FileZilla sono due strumenti essenziali per il trasferimento di file su Internet. FTP è un protocollo standard utilizzato per il trasferimento di file tra un client e un server su reti TCP/IP, mentre FileZilla è un'applicazione con interfaccia user-friendly che ne consente la facile gestione.

In breve, FTP facilita lo scambio di file tra computer remoti, mentre FileZilla è un software per gestire tali trasferimenti in modo efficiente e sicuro.

Il presente documento rappresenta la relazione dettagliata relativa al progetto di sviluppo di un client e un server FTP in linguaggio C, mirato a garantire la piena compatibilità con uno dei client FTP più diffusi, FileZilla.

Entrambi dovranno rispettare le specifiche del protocollo FTP per garantire interoperabilità tra i diversi sistemi SW (il nostro e quello ufficiale di FileZilla)

Il progetto si propone di fornire una base solida per il trasferimento di file in ambiente FTP, concentrandosi sull'implementazione di una serie ridotta ma essenziale di comandi.

Sebbene il numero di comandi implementati sia limitato, sono progettati per coprire le funzionalità di base (connessione + scambio + navigazione FS) necessarie per la gestione dei trasferimenti.

Tuttavia, è importante sottolineare che il progetto è concepito per essere altamente flessibile e adattabile, consentendo ipoteticamente, l'aggiunta di nuovi comandi e funzionalità per essere sempre più conforme al reale applicativo..

Nel corso di questa relazione, esamineremo approfonditamente le scelte architetturali, le implementazioni specifiche e le sfide incontrate durante lo sviluppo con le relative soluzioni adottate. Analizzeremo le strutture utilizzate, i comandi implementati, nonché il funzionamento generale del server nel gestire le richieste dei client.

Scelte Progettuali

Utilizzo Code Management System (github)

GitHub è estremamente utile in un progetto per diverse ragioni:

- Condivisione del codice:
 - GitHub fornisce una piattaforma centralizzata per ospitare il codice del progetto, consentendo al team di accedere e lavorare sullo stesso codice da qualsiasi luogo.
- Controllo delle versioni:
 - Git, integrato in GitHub, offre un robusto sistema di controllo delle versioni che consente di monitorare le modifiche apportate al codice nel tempo e di ripristinare facilmente versioni precedenti se necessario.
- Collaborazione:
 - Le funzionalità di GitHub come le richieste pull, le revisioni del codice e i problemi facilitano la collaborazione tra i membri, consentendo loro di

esaminare, discutere e integrare le modifiche proposte in modo efficiente e trasparente.

- Gestione dei progetti:
 - GitHub offre strumenti integrati per la gestione dei progetti, come le bacheche dei progetti e i problemi, che consentono ai membri di organizzare e tracciare le attività e le responsabilità del progetto in modo chiaro e strutturato.

Attraverso le funzionalità precedentemente elencate, è possibile favorire un efficiente collaborazione anche per team dislocati.

Modifiche successive

Come già affermato, attualmente questo progetto potrebbe essere considerato come una prima release dotata delle funzioni basilari.

Come studiato in altri corsi, un ipotetico processo a modello iterativo prevede vari rilasci dell'applicativo che, a seguito della ricezione di feedback dagli utenti, subirà modifiche (correttive e non) e aggiunte di funzionalità.

Durante la scrittura della documentazione, ci siamo accorti di alcune possibili migliorie e controlli (non errori funzionali) che porterebbero il progetto ad un livello di completezza tale da rispecchiare in tutto e per tutto quello che è Filezilla.

Active VS Passive

Il protocollo FTP (File Transfer Protocol) supporta due modalità di trasferimento dati: attiva e passiva. Queste modalità definiscono come avviene la connessione della Data Socket tra il client e il server durante il trasferimento di dati.

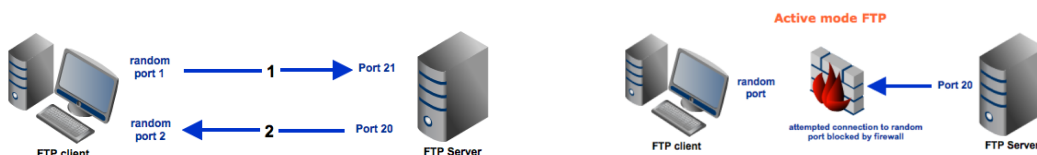
Active

Tra le due modalità di connessione, la modalità attiva è quella più vecchia. L'FTP attivo è stato introdotto agli albori dell'informatica, quando i mainframe erano più comuni e gli attacchi alla sicurezza delle informazioni non erano così diffusi.

Dopo aver istituito la connessione tra Client e Server e aver quindi creato la Command Socket, L'FTP attivo prevede le seguenti operazioni:

- Il client invia il comando PORT, specificando come parametro a quale porta lato client deve connettersi il server (solitamente una porta casuale non privilegiata).
- Questa porta verrà utilizzata in seguito per la Data Socket ed è diversa dalla porta utilizzata per la Command Socket.
- Il server si connette dalla propria porta 20 alla porta client ricevuta come parametro del comando PORT. Una volta stabilita la connessione dati, i trasferimenti di dati avvengono tramite la Data Socket appena istituita.

Questa modalità presenta delle problematiche: quando il client si trova dietro un firewall che blocca le connessioni in ingresso, in questo caso effettuata dal Server, è impossibile istituire la Data Socket.



Codice

Client:

```
void portCommand(struct command *cmd, int server_sock, int *data_socket){
    char retBuff[255];
    int ip[4], portPart[2], port;
    char *response = "%d,%d,%d,%d,%d,%d";
    int enableOpt =1;

    memset(cmd->arg, 0, sizeof(cmd->arg));

    srand(time(NULL));
    portPart[0] = 128 + (rand()%64);
    portPart[1] = (rand()%255);
    port = portPart[0]*256 + portPart[1];

    //prendo IP dalla socket server
    socklen_t addr_size = sizeof(struct sockaddr_in);
    struct sockaddr_in addr;
    getsockname(server_sock, (struct sockaddr *)&addr, &addr_size); //salvo indirizzo formattato dentro addr

    char* host = inet_ntoa(addr.sin_addr);
    sscanf(host,"%d.%d.%d.%d",&ip[0],&ip[1],&ip[2],&ip[3]);

    //filla il messaggio definito in response e lo mette dentro buff
    sprintf(retBuff,response,ip[0],ip[1],ip[2],ip[3],portPart[0],portPart[1]);

    printf("\t%s\n", retBuff);
    printf("\tPorta: %d\n", port);

    strncpy(cmd->arg, retBuff, strlen(retBuff));

    /* Close previous passive socket */
    if(*data_socket > 0){
        close(*data_socket);
    }

    if((*data_socket = socket(AF_INET, SOCK_STREAM, 0)) < 0){
        fprintf(stderr, "Cannot open socket");
        close(*data_socket);
        exit(EXIT_FAILURE);
    }

    /* Server address */
    struct sockaddr_in address;
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(port);

    /* Address can be reused instantly after program exits */
    setsockopt(*data_socket, SOL_SOCKET, SO_REUSEADDR, &enableOpt, sizeof(enableOpt));

    /* Bind socket to server address */
    if(bind(*data_socket,(struct sockaddr*) &address, sizeof(address)) < 0){
        perror("exit:");
        fprintf(stderr, "Cannot bind socket to address\n");
        close(*data_socket);
        exit(EXIT_FAILURE);
    }

    listen(*data_socket,5);

    if(ftclient_send_cmd(cmd, server_sock)!= 0){
        perror("error sending command : ");
    };

    read_reply(server_sock);

    fflush(stdin);

    return;
}
```

inet_ntoa → converte da un numero a 32bit (ovvero la rappresentazione dell'ip in Network order) in una stringa dotted decimal (x.x.x.x)

Le accept per le port vengono fatte nella struttura IF di tutti i comandi.

Server:

```

int handle_PORT_Command(int client_socket, char *bufferIn, Session *state) {
    char ip[4];
    unsigned char p1, p2;

    sscanf(bufferIn, "PORT %hhu,%hhu,%hhu,%hhu,%hhu,%hhu", &ip[0], &ip[1], &ip[2], &ip[3], &p1, &p2);
    int data_port = p1 * 256 + p2;

    state->data_socket = create_data_connection((unsigned char*) ip, data_port);
    if (state->data_socket < 0) {
        snprintf(bufferIn, BUFFER_SIZE, "425 Can't open data connection.\r\n");
        send(client_socket, bufferIn, strlen(bufferIn), 0);
    } else {
        snprintf(bufferIn, BUFFER_SIZE, "200 PORT command successful.\r\n");
        send(client_socket, bufferIn, strlen(bufferIn), 0);
    }
    return state->data_socket;
}

int create_data_connection(unsigned char ip[4], int port) {
    int data_socket = socket(AF_INET, SOCK_STREAM, 0);
    if (data_socket < 0) {
        perror("socket");
        return -1;
    }

    struct sockaddr_in client_addr;
    client_addr.sin_family = AF_INET;
    client_addr.sin_port = htons(port);
    client_addr.sin_addr.s_addr = htonl((ip[0] << 24) | (ip[1] << 16) | (ip[2] << 8) | ip[3]);

    if (connect(data_socket, (struct sockaddr *)&client_addr, sizeof(client_addr)) < 0) {
        perror("connect");
        close(data_socket);
        return -1;
    }
    return data_socket;
}

```

“al posto di htonl e dello shift binario avremmo potuto usare delle funzioni come inet_addr() o inet_aton() che prendendo in input una stringa del formato di un IP (x.x.x.x) la convertono in un numero a 32 bit in Network Byte Order”

Passive

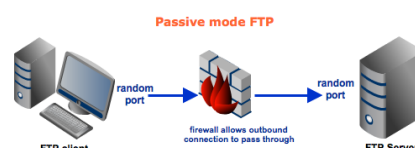
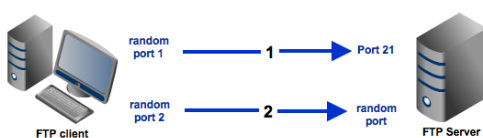
Anche nella modalità passiva è il Client che dà il via alla creazione della Data Socket.

Dopo aver istituito la connessione tra Client e Server e aver quindi creato la Command Socket, L'FTP passivo prevede le seguenti operazioni:

- Il Client invia il comando PASV per comunicare al Server di voler aprire una Data Socket
- Il server risponde, indicando quale porta (casuale) ha aperto per il trasferimento dei dati.
- Il client si connette da un'altra porta casuale alla porta specificata nella risposta del server. Una volta stabilita la connessione, i trasferimenti di dati vengono effettuati sulla Socket appena istituita.

La modalità passiva è generalmente preferita in situazioni in cui il client si trova **dietro un firewall o NAT**, poiché consente al client di iniziare la connessione dati verso il server senza problemi di inoltro dei firewall.

Tuttavia, entrambe le modalità sono supportate e possono essere utilizzate in base alla configurazione del server e alle esigenze di sicurezza del sistema.



Codice

Server:

```
int handle_PASV_Command(int client_socket, char *bufferIn, Session *state) {
    int ip[4];
    int portPart[2], port;
    char buff[255];
    char *response = "227 Entering Passive Mode (%d,%d,%d,%d,%d,%d)\n";

    struct sockaddr_in client_addr;
    socklen_t addr_len = sizeof(client_addr);
    int dataServer;

    srand(time(NULL));
    portPart[0] = 128 + (rand()%64);
    portPart[1] = (rand()%255);
    port = portPart[0]*256 + portPart[1];

    /* Close previous passive socket */
    close(state->data_socket);

    /* Start listening here, but don't accept the connection */
    dataServer = create_socket_pasv(port);

    //prendo IP dalla socket client
    socklen_t addr_size = sizeof(struct sockaddr_in);
    struct sockaddr_in addr;
    getsockname(client_socket, (struct sockaddr *)&addr, &addr_size); //salvo indirizzo formattato dentro addr

    char* host = inet_ntoa(addr.sin_addr);
    sscanf(host, "%d.%d.%d.%d", &ip[0], &ip[1], &ip[2], &ip[3]);

    //filla il messaggio definito in response e lo mette dentro buff
    sprintf(buff, response, ip[0], ip[1], ip[2], ip[3], portPart[0], portPart[1]);

    printf("\t%s", buff);

    send(client_socket, buff, strlen(buff), 0);

    if ((state->data_socket = accept(dataServer, (struct sockaddr *)&client_addr, &addr_len)) < 0) {
        perror("error accept");
        exit(EXIT_FAILURE);
    }
    return 0;
}
```

```
int create_socket_pasv(int port){
    int sock;
    int reuse = 1;

    if((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0){
        fprintf(stderr, "Cannot open socket");
        exit(EXIT_FAILURE);
    }

    /* Server address */
    struct sockaddr_in address;
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(port);

    /* Address can be reused instantly after program exits */
    setsockopt(sock, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof reuse);

    /* Bind socket to server address */
    if(bind(sock, (struct sockaddr *)&address, sizeof(address)) < 0){
        perror("exit:");
        fprintf(stderr, "Cannot bind socket to address\n");
        exit(EXIT_FAILURE);
    }

    listen(sock, 5);
    return sock;
}
```

Client:

```

void passive(int server_sock, int *data_socket){
    char retBuff[1024];
    int retCode=-1;
    int ip[4], port[2], passivePort; //ho aggiunto la passivePort per le robe che aggiungo alla funzione in fondo
    char ip_port[27];
    char *token;

    if (recv(server_sock, &retBuff, sizeof retBuff, 0) < 0) {
        perror("\terror read reply: ");
        return;
    }

    //retBuf contiene la risposta
    printf(MAGENTA);
    printf("\t%s", retBuff);
    printf(RESET);

    //ret code
    sscanf(retBuff, "%d", &retCode);

    //spezzo la stringa in token per tenere l'ultimo con ip e porta
    //dato che all'ultimo ciclo il token sarà a NULL mi salvo il valore in un'altra variabile
    token = strtok(retBuff, " ");
    while( token != NULL ) {
        strcpy(ip_port, token);
        token = strtok(NULL, " ");
    }

    sscanf(ip_port, "%s[^0-9]%d%[^0-9]%d%[^0-9]%d%[^0-9]%d%[^0-9]%d%[^0-9]%d", &ip[0], &ip[1], &ip[2], &ip[3], &port[0], &port[1]);

    passivePort = port[0] * 256 + port[1];

    struct sockaddr_in server_addr;

    if ( (*data_socket = socket(AF_INET, SOCK_STREAM, 0)) <= 0 ){
        perror("Could not create socket");
        exit(EXIT_FAILURE);
    }

    server_addr.sin_family = AF_INET;
    server_addr.sin_port = htons(passivePort);
    server_addr.sin_addr.s_addr = htonl((ip[0] << 24) | (ip[1] << 16) | (ip[2] << 8) | ip[3]);

    // Connect to server_addr
    if (connect(*data_socket, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
        perror("Connection failed");
        exit(EXIT_FAILURE);
    }
}

```

Firewall

Per consentire il passaggio dei pacchetti in ingresso verso un PC che funge da server all'interno della tua rete locale, è necessario aprire le porte del firewall per il servizio specifico che si desidera offrire.

Se si sta eseguendo un server FTP, come discusso in precedenza, si dovranno aprire le porte FTP (21 per il controllo e 20 per il trasferimento dati) nel firewall del server.

Come abbiamo analizzato [prima](#): la modalità attiva presenterebbe dei problemi, in quanto la connessione avviene anche dal Server al Client (port), sarebbe quindi necessario aprire la porta su cui il Client attende la connessione del Server ma, dato che viene generata randomicamente, si dovrà disattivare il Firewall (non molto consigliato).

► SERVER

Un server FTP (File Transfer Protocol) è un'applicazione o un dispositivo hardware che utilizza il protocollo FTP per consentire a client remoti di accedere e trasferire file da e verso il server su una rete TCP/IP. Il server FTP funge da punto centrale per l'archiviazione e la gestione dei file, consentendo agli utenti di accedere ai file memorizzati e di eseguire operazioni di upload, download, eliminazione e altre operazioni di gestione dei file.

Il funzionamento di base di un server FTP consiste nell'accettare le connessioni dai client che desiderano accedere ai file. Una volta stabilita una connessione, il client può autenticarsi utilizzando un nome utente e una password se richiesti dal server. Una volta autenticato, il client può eseguire una varietà di comandi FTP per eseguire operazioni sui file, come ad esempio elencare i file disponibili, trasferire file da e verso il server, creare e eliminare directory e altro ancora.

Il server FTP gestisce queste operazioni accettando e rispondendo ai comandi inviati dal client, e inoltrando le richieste di trasferimento dei file attraverso una connessione dati separata. Questo processo consente ai client di trasferire i dati in modo affidabile e efficiente, indipendentemente dalla complessità o dalla quantità dei file coinvolti.

In sintesi, un server FTP fornisce un'infrastruttura centrale per il trasferimento di file su una rete TCP/IP, consentendo agli utenti di accedere e gestire i file memorizzati nel server attraverso l'uso del protocollo FTP.

Strutture utilizzate

Commands

La struttura fondamentale su cui si basa il server è la struttura Commands, così costituita:

```
typedef struct
{
    char *name;
    int (*commandFunc)(int ,char*,Session*);
}COMMANDS;
```

- *name*
 - il nome del comando
- *commandFunc*
 - un puntatore a funzione
 - la funzione ritorna un intero e prende come parametri:
 - la Client Socket (creata nel main e passata al thread)
 - un buffer
 - una struttura Session contenente informazioni generali della sessione/connessione

Session

Struttura che definisce il tipo di un parametro passato a tutti gli handler.

Questo parametro si occupa di tenere traccia dell'utente (usr+pwd), mantenere i FD della Data Socket e infine tiene traccia della corrente cartella di lavoro per tenere traccia delle posizioni del File System del server.

```
typedef struct
{
    char username[256];
    char password[256];
    int data_socket; //socket per scambio dati (retr/store/list)
    char current_working_dir[2048];
}Session;
```


Array COMMANDS

Utilizziamo *COMMANDS* nel main per creare un array di strutture (mComands) di questo tipo:

```
COMMANDS mComands[] = {
    {"USER", handle_USER_Command}, {"PASS", handle_PASS_Command},
    {"PORT", handle_PORT_Command}, {"LIST", handle_LIST_Command},
    {"PWD", handle_PWD_Command}, {"CWD", handle_CWD_Command},
    {"DELE", handle_DELE_Command}, {"RETR", handle_RETR_Command},
    {"STOR", handle_STOR_Command}, {"MKD", handle_MKD_Command},
    {"RMD", handle_RMD_Command}, {"TYPE", handle_TYPE_Command},
    {"QUIT", handle_QUIT_Command}, {"PASV", handle_PASV_Command}
};
```

Ricerca/Esecuzione handler

```
mComands[i].commandFunc(client_fd, buffer, &state);
```

Viene eseguito un ciclo che itera sull'array di strutture mComands facendo una *strcasecmp* tra il comando ricevuto e il campo *name* della struttura per identificarne la posizione all'interno dell'array.

```
if (isValidCommand(cmd) != -1) { //fa controllo nell'array di comandi
    for (int i = 0; i < NUMBER_OF_COMMANDS; ++i) {
        if (strcasecmp(cmd, mComands[i].name) == 0) { //cerca se il comando inviato dall'utente corrisponde a uno presente
            mComands[i].commandFunc(client_fd, buffer, &state);
            break;
        }
    }
}
```

Una volta identificato l'indice di interesse, invoco la funzione del comando necessario accedendo al campo *commandFunc* come se fosse il metodo di un oggetto (è possibile perché è un puntatore a funzione).

Funzionamento

In questo paragrafo analizziamo prima quello che è la creazione e inizializzazione della Command socket, per poi passare a come viene gestita ogni operazione.

- Creazione Socket server

Illustriamo la sequenza di operazioni da svolgere per la creazione di una socket server:

- socket()
 - Restituisce un file descriptor che rappresenta la socket e viene utilizzato nelle successive operazioni
- ```
// Create a socket
if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
 perror("socket");
 exit(EXIT_FAILURE);
}
```
- AF\_INET → address family → dominio
    - SOCK\_STREAM → socket type (Provides sequenced, reliable, bidirectional) → associato a TCP (sock\_dgram per UDP)
  - bind()
    - Viene utilizzata per assegnare un indirizzo IP e una porta specifici a una socket.

```
// Bind the socket
if (bind(server_fd, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
 perror("bind");
 exit(EXIT_FAILURE);
}
```

- 
- listen()

- Viene utilizzata su una socket di tipo server per segnalare che la socket è pronta per accettare connessioni in arrivo. (Utilizzabile solo per TCP, no UDP in quanto UDP non crea una connessione)

```
// Listen for connections
if (listen(server_fd, 5) < 0) {
 perror("listen");
 exit(EXIT_FAILURE);
}
```

- 

- il secondo parametro indica il numero di client massimi in coda (attesa di essere accettati → se non fosse multithread si saturerebbe la coda velocemente)

- accept()

- Viene utilizzata per accettare una connessione in arrivo su una socket di tipo server. Restituisce una nuovo socket che rappresenta la connessione accettata. Questa nuova socket viene utilizzata per la comunicazione con il client specifico (Utilizzabile solo per TCP, no UDP in quanto UDP non crea una connessione)

```
// Accept a connection
if ((client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len)) < 0) {
 perror("accept");
 exit(EXIT_FAILURE);
}
```

- 

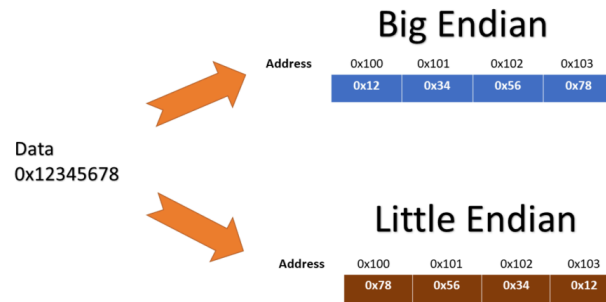
- è una funzione bloccante, rimane in attesa finché non accetta
- il secondo e terzo parametro sono due strutture vuote riferite all'indirizzo del client, verranno inizializzate quando l'accept ritorna
- il client, quando esegue la connect, avvia un'operazione di *3-way-handshake* al termine della quale si conclude la accept (lato server) e la connect (lato client) stabilendo la connessione

## Conversioni

Prima di invocare la bind(), per la configurazione della struttura sockaddr\_in, andiamo ad utilizzare la funzione htons().

htons(): permette di convertire un valore in network byte order, quindi in Big Endian.

- **Host byte ordering:** L'ordine dei byte dipende dall'architettura del computer (*big endian* o *little endian*).
- **Network byte ordering:** L'ordine dei byte è sempre *big-endian* (il byte più significativo viene memorizzato prima).



#### Funzioni di conversione per short(2bytes) e long(4bytes) int

- `localshort = ntohs(netshort)` **n->network, h->host**
- `locallong = ntohl(netlong)`
- `netshort = htons(localshort)`
- `netlong = htonl(locallong)`

La struttura `sockaddr_in` è costituita nel seguente modo:

```
Address format
...
struct sockaddr_in {
 sa_family_t sin_family; /* address family: AF_INET */
 in_port_t sin_port; /* port in network byte order */
 struct in_addr sin_addr; /* internet address */
};
```

L'utilizzo nel nostro progetto per l'assegnazione di una porta ad una socket:

```
server_addr.sin_port = htons(port);
```

L'indirizzo (IP + Porta) potrà essere riutilizzato immediatamente dopo la terminazione del programma nel seguente modo:

```
/* Address can be reused instantly after program exits */
setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &enableOpt, sizeof(enableOpt));
```

Senza l'utilizzo della funzione `setsockopt`, quando invochiamo la funzione `bind` (in seguito ad un previo utilizzo) potremmo ricevere il seguente errore: *Address already in use*.

#### - Struttura generale Server

Dopo la creazione della Command Socket, il Server ha terminato il suo lavoro, fin quando un client non farà una richiesta di connessione.

Sarà compito del server accettare la richiesta di connessione e cedere al thread la gestione della sessione del singolo Client.

`accept()` + `pthread_create()`

Essendo un Server FTP generalmente ideato per essere multi-thread, per svilupparlo rispecchiando il più possibile la struttura di Filezilla, abbiamo deciso di utilizzare la programmazione concorrente per la gestione di più Client.

```
while (1) {
 // Accept a connection
 if ((client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &addr_len)) < 0) {
 perror("accept");
 exit(EXIT_FAILURE);
 }
}
```

Come struttura generale dei Server, abbiamo un ciclo infinito di accettazione di nuove connessioni; attraverso la funzione `accept` ci viene restituito il File Descriptor per la socket (*command*) di ogni utente.

```
pthread_t clientThread;
pthread_create(&clientThread, NULL, handle_client, (void *)client_fd);
}
```

Una volta ottenuto il File Descriptor la gestione del client viene delegata al thread (*handle\_client*) a cui passiamo come parametro il FD della socket che sta gestendo. La `create` prende in ingresso la funzione da eseguire e, eventualmente, i parametri della funzione passati come puntatore `void` (poiché "universale" e richiesto dal tipo di parametri della `create`).

Per evitare warning del compilatore, abbiamo deciso di salvare nel Main il FD delle socket client in una variabile `long`, ciò causerà "solo" uno spreco di memoria, in quanto quest'ultimo è un `int` (ma rimuove i warning di `pointer-to-int-cast`).

Dopo aver passato il FD al thread come (*void \**), sarà poi necessario castarlo nuovamente a `long`. Per correttezza l'abbiamo poi salvato in una variabile `int` (tanto non perdo dati utili in quanto il FD è `int`).

```
void *handle_client(void *client_fdIn) {
 int client_fd = (long)client_fdIn;
 printf("Client File Descriptor %d \n", client_fd);
}
```

### Thread: *handle\_client*

Riga 116 *ftpServer.c*

- setto una *CurrentWorkingDir* con *getenv* (la home)
- setto quei valori nello *state* (variabile di tipo *struct Session*)
- Invio *welcome message* al client (serve per conferma connessione al client)
- while attesa ricezione comandi (quando *recv > 0* → è arrivato un comando)
  - *isValidCommand* → controlla che sia un comando presente nell'implementazione del server (semplice *strcmp*)
  - itero sull'array di strutture *mComands* facendo una *strcmp* tra il comando ricevuto e il campo *name* della struttura per identificare la posizione all'interno dell'array.
  - una volta identificato l'indice di interesse, invoco la funzione del comando necessario accedendo al campo *commandFunc* come se fosse il metodo di un oggetto (è possibile perché è un puntatore a funzione).
    - `mComands[i].commandFunc(client_fd, buffer, &state);`

Da qui la gestione di ogni comando viene delegata agli opportuni handler.

### Handlers

- USER
  - Viene controllato se il nome utente è corretto e viene inviata una risposta al client:
    - Se è corretto
      - 331 User name okay, need password.
    - Se è sbagliato
      - 530 Not logged in.
- PASS
  - Viene controllato se la password è corretta e viene inviata una risposta al client:
    - Se è corretto
      - 230 User logged in, proceed.
    - Se è sbagliato
      - 530 Not logged in.
- PORT
  - Prendiamo i valori di IP e Porta (inviati come parametri) e tramite la funzione *create\_data\_connection* andiamo a creare la Data Socket collegandoci direttamente al Client → *connect()* (Il server in questo caso si comporta come un Client)
  - Viene poi inviato un messaggio al Client in base al successo della connessione:
    - Se va a buon fine
      - 200 PORT command successful.
    - Se c'è un errore
      - 425 Can't open data connection.
- PASV
  - Dopo aver generato randomicamente una porta e aver chiuso la Data Socket precedente (se esiste), viene creata una socket in ascolto tramite la funzione *create\_socket\_pasv* che restituisce il FD di quest'ultima.

- Quando arriva la richiesta dal Client, viene fatta la *accept* e creata la Data Socket.
- LIST
  - Prova ad aprire la directory corrente e restituisce un messaggio:
    - Se va a buon fine
      - 150 data connection for file list.
    - Se c'è un errore
      - 550 Failed to open directory.
  - In caso di successo, la directory viene letta tramite la funzione *readdir* da cui vengono poi estrapolate le informazioni, ad esempio i permessi (Simile al comando *ls -la* di Linux) e viene restituito il messaggio di completamento:
    - 226 Transfer complete.
- PWD
  - Il Server invia al Client la directory corrente di lavoro .
    - 257 <dir> is the current directory.
- CWD
  - Cambia la directory di lavoro invocando una funzione di SO (*chdir*) e settando il cambiamento nello state.
  - Viene restituito il messaggio di completamento:
    - 250 Directory successfully changed to <dir>.
- RMD
  - Rimuove la directory selezionata e restituisce un messaggio:
  - Se va a buon fine
    - 250 Directory removed successfully.
  - Se c'è un errore
    - 550 Failed to remove directory.
- MKD
  - Crea una directory con il nome selezionato e restituisce un messaggio:
    - Se va a buon fine
      - 250 Directory created successfully.
    - Se c'è un errore
      - 550 Failed to create directory.
- RETR
  - In un while legge chunk di bytes (BUFFER\_SIZE) dal File richiesto e li invia al client fino al raggiungimento di EOF
  - Invia un messaggio di completamento:
    - 226 Transfer complete.
- STOR
  - In un while legge chunk di bytes (BUFFER\_SIZE) dalla Socket Data e li scrive direttamente nel file fino a quando la *recv* ritorna un valore  $\leq 0$
  - Invia un messaggio di inizio invio e di completamento:
    - 150 Ok to send data.
    - 226 Transfer complete.
- DELE
  - Rimuove il file selezionato e restituisce un messaggio:
    - Se va a buon fine
      - 250 File deleted successfully.
    - Se c'è un errore

- 550 Failed to delete file.
- TYPE
  - permette di settare il tipo a Ascii o Binary
  - Invia un messaggio in base al tipo settato:
    - Se Binary
      - 200 Type set to I.
    - Se Ascii
      - 200 Type set to A.
    - Errore
      - 504 Command not implemented for that parameter.
- QUIT
  - invia il messaggio di saluto e chiude la connessione
    - 221 Goodbye.

## Logging

L'operazione di scrittura dei [log](#) dei comandi avviene tramite un thread a sé stante.

- Quando *countCommands* (variabile per contare il numero di comandi ricevuti) vale 9, non si entra nel while che manda *handle\_client* in attesa della funzione di *log* (la *wait cond empty*)

```
pthread_mutex_lock(&mutex);
while(countCommands >= 10)
{
 printf("Wait for new command\n");
 pthread_cond_wait(&cond_empty,&mutex);
}
strcpy(logs.command[countCommands],cmd);
logs.state = &state;
countCommands++;
pthread_mutex_unlock(&mutex);
if(countCommands == 10)pthread_cond_signal(&cond_full);
```

- 
- l'istruzione seguente è un incremento del contatore che provoca così l'ingresso nell'if in quando countCommands=10
  - facendo così si segnalerà al thread di logging di attivarsi e di scrivere sul file;
  - una volta terminata quest'operazione viene svolta una signal che riattiva *handle\_client* che nel frattempo è entrato nel while di attesa

```
//Funzione Thread di logging --> ogni 10 comandi (gestito da mutex)
void *log_Message(){
 FILE * fp = fopen("logs.txt","a");
 if(fp == NULL)printf("Problem Opening File\n");
 while(1)
 {
 pthread_mutex_lock(&mutex);
 while(countCommands < 10)
 {
 pthread_cond_wait(&cond_full,&mutex);
 }
 for(int i =0; i < countCommands;++i)
 {
 fprintf(fp, "[%s] : %s\n",logs.state->username ,logs.command[i]);
 fflush(fp);
 }
 countCommands = 0;
 pthread_mutex_unlock(&mutex);
 pthread_cond_signal(&cond_empty);
 }
}
```

## Scelte

### Server multithread

Abbiamo scelto di implementare un server multi-thread anziché single-thread per garantire una maggiore efficienza e scalabilità nel gestire le richieste degli utenti. Con un server multi-thread, ogni richiesta di connessione viene gestita da un thread separato, consentendo al server di gestire più connessioni contemporaneamente senza interruzioni.

Inoltre, un server multi-thread offre una maggiore affidabilità in quanto le richieste dei clienti non sono bloccate da altre attività in attesa di essere elaborate. Questo significa che anche se un thread dovesse riscontrare un problema o essere occupato con un'operazione lunga, gli altri thread possono continuare a gestire le richieste degli altri clienti senza alcun impatto sulle prestazioni complessive del server.

In sintesi, l'implementazione di un server multi-thread offre maggiore efficienza, scalabilità e affidabilità rispetto a un approccio single-thread.

### Implementazione logging (nel server)

L'implementazione del logging in un server FTP è essenziale per diversi motivi:

- Tracciabilità delle attività:
  - Il logging registra tutte le attività svolte sul server FTP, consentendo di tracciare chi ha accesso al server e quali operazioni vengono eseguite. Questo è fondamentale per scopi di sicurezza e audit.
- Monitoraggio delle prestazioni:
  - Il logging può essere utilizzato per monitorare le prestazioni del server FTP, tenendo traccia di metriche come il tempo di risposta (da implementare), il numero di connessioni, il traffico di rete e altro ancora. Queste informazioni sono utili per identificare eventuali problemi di prestazioni e ottimizzare le risorse del server.
- Conformità normativa:
  - In molti contesti, è richiesto per legge o per motivi di conformità normativa che vengano mantenuti registri dettagliati delle attività svolte su un server FTP. Il logging aiuta a soddisfare questi requisiti fornendo una traccia completa delle operazioni eseguite.

### Gestione struct con puntatori a funzione

In linguaggio C, l'utilizzo di puntatori a funzione all'interno di strutture può essere una tecnica utile per implementare una forma rudimentale di **polimorfismo e modularità** del codice.

La struttura prevede l'utilizzo di una struct contenente il nome del singolo comando e un puntatore alla funzione che lo riguarda, questo per far sì che, qualora si dovesse aggiungere un comando, basterà inserire un elemento nell'array di struct che gestisce i comandi.

[Vedi per maggiori dettagli → Strutture Utilizzate](#)

## Credenziali

Credenziali di login per collegarsi col nostro server:

USERNAME: user  
PASSWORD: segreta



## ► CLIENT

Un client FTP (File Transfer Protocol) è un'applicazione software progettata per consentire agli utenti di accedere e gestire i file memorizzati su un server FTP remoto attraverso una connessione di rete. In sostanza, funge da interfaccia utente che consente agli utenti di eseguire operazioni di upload, download e gestione dei file sul server FTP.

Il funzionamento di base di un client FTP consiste nel connettersi al server FTP utilizzando un indirizzo IP o un nome di dominio e specificando le credenziali di accesso, come il nome utente e la password. Una volta autenticati, gli utenti possono navigare attraverso le directory del server, visualizzare i file disponibili e interagire con essi tramite una serie di comandi FTP.

I principali compiti eseguiti da un client FTP includono:

- Connessione al server FTP: Il client FTP stabilisce una connessione di rete con il server FTP utilizzando il protocollo FTP.
- Autenticazione: Gli utenti forniscono le proprie credenziali di accesso (nome utente e password) per autenticarsi presso il server FTP e accedere ai file.
- Trasferimento di file: Gli utenti possono trasferire file da e verso il server FTP utilizzando comandi come STOR (Upload) e RETR (Download).
- Gestione dei file: Il client FTP consente agli utenti di eseguire operazioni di gestione dei file, come creare, spostare ed eliminare file e directory sul server FTP.
- Navigazione delle directory: Gli utenti possono navigare attraverso le directory del server FTP per individuare i file desiderati e visualizzare la struttura delle cartelle.

## Funzionamento

### - Creazione Socket Client

Illustriamo la sequenza di operazioni da svolgere per la creazione di una socket client:

- `socket()`
  - Restituisce un file descriptor che rappresenta la socket e viene utilizzato nelle successive operazioni

```
if ((server_sock = socket(AF_INET, SOCK_STREAM, 0)) == 0){
 perror("Could not create socket");
 exit(EXIT_FAILURE);
}
```

- `connect()`
  - si connette al server (indirizzo e porta) specificato

```
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(atoi(argv[1]));
if(inet_pton(AF_INET, "192.168.178.140", &server_addr.sin_addr) <= 0) {
 perror("Invalid address/Address not supported");
 exit(EXIT_FAILURE);
}

// Connect to server_addr
if(connect(server_sock, (struct sockaddr *)&server_addr, sizeof(server_addr)) < 0) {
 perror("Connection failed");
 exit(EXIT_FAILURE);
}
```

- viene eseguito un *3-way-handshake* al termine del quale viene stabilita la connessione (va a buon fine la connect lato client e la accept lato server)
- *non usabile in UDP in quanto non viene stabilita una connessione*

### - Struttura generale Client

Il client prevede che venga creata una Socket e che venga effettuata una connessione al Server FTP, dopo il messaggio di benvenuto del Server verrà effettuata l'autenticazione e l'utente potrà interagire col Server con i comandi FTP.

```
read_reply(server_sock);
ftclient_login(server_sock);
int active = 0;
while (1) { // loop until user types quit
 // Get a command from user -> il (comando+args) è dentro buffer mentre cmd è fillato con i suoi parametri
 if (ftclient_read_command(buffer, sizeof buffer, &cmd) < 0) {
 // printf("Invalid command\n");
 continue; // loop back for another command
 }
}
```

1. creazione socket e connessione
2. *read\_reply*
  - //ritorna il codice della risposta del server (retcode)
  - stampa alcuni dei messaggi inviati dal server
  - usata per messaggio di benvenuto
3. *ftclient\_login*
  - invia comando USER <username>
  - se va a buon fine (lettura *read\_reply*) invia PASS <pwd>
  - (utilizziamo credenziali preimpostate di testing)
  - utilizziamo ancora *read\_reply* per stampare stato
4. while di gestione/invio comandi
5. *ftclient\_send\_cmd*
  - funzione che invia ogni comando valido al server attraverso la Command Socket (viene utilizzata nei vari IF del while spiegato qui di seguito)

### Struttura While

1. *ftclient\_read\_command*
  - prende un comando (cmd+args) dall'utente e "fillo" le opportune strutture per la gestione dei comandi
  - *isValidCommand*
2. serie di IF per la selezione del codice da eseguire in base al comando
  - viene gestita da un flag l'esecuzione *attiva* o *passiva*
3. *ftclient\_send\_cmd*

### Gestione comandi (dentro while)

- PORT
  - Il Client invia al Server il proprio IP e una porta scelta randomicamente al server sulla **Command Socket** (socket generale del server).

- Il Server si connette al Client dalla propria porta 20 all'ip e alla porta comunicata da quest'ultimo → creazione **Data Socket**
  - Il client deve aver "bindato" una socket per attendere la connessione in ingresso del server
- tramite `read_reply` stampiamo l'esito dell'operazione
- **STOR**
  - Il Client invia sulla *Command Socket*, attraverso la funzione `ftclient_send_cmd`, il comando (STOR <parametro>) per avvisare il server del caricamento
  - Utilizzando poi la *Data Socket*, subito dopo l'ok di inizio trasferimento da parte del server, il Client invia (send) al Server un file che quest'ultimo archiverà
- **RETR**
  - Il Client invia sulla *Command Socket*, attraverso la funzione `ftclient_send_cmd`, il comando (RETR <parametro>) per avvisare il server dello scaricamento.
  - Utilizzando poi la *Data Socket*, subito dopo l'ok di inizio trasferimento da parte del server, il Client inizia a leggere (read) i contenuti inviati dal server e a trasferimento completato archivia il file.
- **PASV**
  - Il Client riceve un IP e una porta (in ascolto) dal server sulla *Command Socket*
  - Sarà compito del client istanziare una connessione verso quelle specifiche per "aprire" una *Data Socket*.
- **LIST**
  - invia il comando sulla *Command Socket*
  - riceve i dati inviati dal server sulla *Data Socket* (potrebbero essere inviati più messaggi poiché di grandi dimensioni)
  - riga 439 → utilizzo `realloc` per unire tutti i buffer letti con le varie `recv` in caso di liste lunghe di file
    - vengono fatte una serie di `recv` in un ciclo finché si è letto tutto
- **QUIT**
- tutti gli altri comandi
  - Per i comandi che non richiedono l'utilizzo della *Data Socket*, il messaggio viene semplicemente inviato e viene letta la risposta del Server.
    - Ad esempio: PWD, CWD <dir> ...

## Gestione Active VS Passive

L'utilizzo dei comandi che richiedono trasferimento di dati sulla Data Socket (list, retr, stor) prevedono l'invocazione dei comandi PASV/PORT per la creazione della socket; nel caso in cui venisse richiesto un tale comando senza soddisfare i prerequisiti il comando non andrà a buon fine invitando l'utente ad un corretto utilizzo.

La gestione di attivo e passivo avviene tramite l'utilizzo di un flag denominato `active` (inizializzato a 0) che viene impostato a 1 nel momento in cui viene richiamata la funzione `PORT`.

La gestione comandi che richiedono l'utilizzo della Data Socket (list, retr, stor), prevede che venga controllato il flag e:

```
else if((strcasecmp(cmd.code,"RETR")) == 0){
 if(cmd.arg[0] == '\0'){
 printf("\tE' necessario inserire il nome di un file!\n");
 }else{
 if(ftclient_send_cmd(&cmd, server_sock) != 0){
 perror("error sending command : ");
 };
 if(active){
 data_sock = acceptPort(data_sock);
 }
 retr(&cmd, server_sock, data_sock);
 active = 0;
 }
}
```

- Qualora sia a 1 (Connessione attiva) viene richiamata la funzione *acceptPort* che effettua una Accept della connessione in ingresso dal Server al Client permettendo quindi di avere una Data Socket funzionante (*vedi port*).
- Qualora sia a 0 (Connessione passiva), verrà utilizzata la Data Socket inizializzata e creata dalla chiamata precedente di PASV.

## Problematiche

### Buffer overflow per LIST

Questa problematica si verificava solo attraverso l'utilizzo del comando List, in quanto la lunghezza della risposta del Server è direttamente proporzionale al contenuto della directory. Talvolta, essendo la lunghezza della risposta maggiore della grandezza del Buffer, non basta l'utilizzo di una sola *recv* bensì diventa necessario effettuare una serie di letture.

(numero di *recv* = quantità ricevuta / *dim\_buffer* → vedo in quante volte riesco a leggere il contenuto totale).

Il problema di queste letture in serie è che tutte le celle della dimensione del buffer vengono occupate dai caratteri ricevuti, è quindi necessaria un'operazione di formattazione: vengono presi tutti i caratteri contenuti nel buffer e concatenati ad una stringa dinamica (inizialmente vuota) che accresce la sua dimensione dopo ogni lettura (così da poter contenere il messaggio indipendentemente dalla dimensione).

Quando non si ha più nulla da leggere (*recv()* <= 0) viene messo il terminatore di stringa (*\0*) e si può utilizzare il messaggio ricevuto

### Messaggio singolo/doppio

Durante lo sviluppo, tramite l'utilizzo del debugger, abbiamo notato comportamenti anomali nei comandi LIST e RETR in merito all'invio dei messaggi da parte del Server sulla Command Socket (inizio e conferme).

Difatti, il server in alcuni casi invia il messaggio di inizio trasferimento (*150 Starting data transfer*) insieme a quello di completamento (*226 Operation successful*) mentre in altri invia i due messaggi in modo separato; questo comporta rispettivamente l'utilizzo di una *recv* e di due *recv* per leggere i messaggi.

Per risolvere questa anomalia abbiamo semplicemente posto un controllo sul contenuto del buffer della prima *recv*: se questo contiene anche "226" allora abbiamo ricevuto entrambi i messaggi di risposta del Server, in caso contrario è necessario effettuare un'altra ricezione.

## Credenziali

Credenziali di login sul server filezilla di windows:

USERNAME: test

PASSWORD: test